Programming 2

Homework 2

Contents

Code of Conduct 2
Overview 2
Requirements 3
ConstantValues 3
PersonID 4
PersonID – attributes 4
PersonID - methods 4
Course 6
Course – attributes 6
Course - constructors 6
Course - methods 6
StudentCourse 8
StudentCourse – attributes 8
StudentCourse - constructors 8
StudentCourse - methods 8
Degree 10
Degree – attributes 10
Degree - methods 10
Student 12
Student – attributes 12
Student - constructors 12
Student – methods (new methods highlighted below) 13
Assessment and Grading 15
Expected output - tests 15
Student – main program 16
Student – main program – missing credits 20

Code of Conduct

Do your homework by yourself, so produce your own material (do not copy from a friend or directly from some source). Completing the homework requires independent adoption of the issues covered in lectures, material and exercises.

If two or more students have identical code or there is suspected misconduct, the issue...

- Will be reported to the education dean of the faculty and/or degree program director
- Can lead to a lower grade (reduction up to 100% of the total points, even if a student's task was already graded)
- Can lead to an automatic failure of the course (even if a student's task was already graded).

Overview

The functionality of the previous, first task has been split to different classes. The task includes a total of 6 classes **ConstantValues**, **PersonID**, **Course**, **StudentCourse**, **Degree**, **Student** - and the Main class for testing.

The idea is to practice the use of **arrays** as well as **association**, **aggregation**, and **composition** plus, of course, the use of **control structures** and **formatting** of texts.

- Implement the class one method at a time, iteratively (incrementing the functionality).

- As before…
  - Always use ***private*** visibility modifier when there is no justified reason to do otherwise.
  - Note types of the attributes.
  - Pay attention to the required default values.
  - Pay attention to the required constructors.
  - Pay attention to the required methods (names, return types, parameters, visibility modifiers & required functionality)
- Study and learn how to use arrays
- Study and learn how to use reference attributes

- Test your class
  - Check your program output matches the expected output for the program

Requirements

ConstantValues

```
public class ConstantValues {

    public static final String NO_NAME = "No name";
    public static final String NO_TITLE = "No title";
    public static final String NO_BIRTHDATE = "Not available";
    public static final String INVALID_BIRTHDAY = "Invalid birthday!";
    public static final String INCORRECT_CHECKMARK = "Incorrect check mark!";

    public static final int MIN_ID = 1;
    public static final int MAX_ID = 100;
    public static final double MIN_CREDITS = 0.0;
    public static final double MAX_CREDITS = 300.0;
    public static final double BACHELOR_CREDITS = 180.0;
    public static final double MASTER_CREDITS = 120.0;

    public static final double MAX_COURSE_CREDITS = 55.0;
    public static final int MIN_PERIOD = 1;
    public static final int MAX_PERIOD = 5;
    public static final int BACHELOR_TYPE = 0;
    public static final int MASTER_TYPE = 1;
    public static final int OPTIONAL = 0;
    public static final int MANDATORY = 1;
    public static final int ALL = 2;

    public static final int MIN_GRADE = 0;
    public static final int MAX_GRADE = 5;
    public static final char GRADE_FAILED = 'F';
    public static final char GRADE_ACCEPTED = 'A';
}
```

PersonID

PersonID – attributes

Attribute name	Type	Description / Notes
birthDate	String	Date of birth in the format dd.mm.yyyy. By default, “Not available”.

PersonID - methods

Method name	Return type	Parameters	Description / Notes
getBirthDate	String	-	Returns the birthDate, by default “Not available”.
setPersonID	String	final String personID	The method will receive a social security number as an input string. The method… a) will check the personID has been given the correct way (use the method **checkPersonIDNumber**) and **if** done so, b) will form the given personID to the format **dd.mm.yyyy** and ensures the values in that date are correct (use the method **checkBirthdate**) and **if** so, c) will check the check character in the original, given personID was correct (use the method **checkValidCharacter** here). If… • all is ok the method will set the birthdate (check the format) and returns “Ok”. • personID does not have 11 characters and the “century character is not one of the valid ones (‘+’, ‘-‘ or ‘A’), based on the return value of method checkPersonIDNumber, the method will return "Invalid birthday!". • the days, months or year in the given person id are not correct considering the month or year (note leap year), based on the return value of checkBirthdate, the method will return "Invalid birthday!" • the last character in the person id (the check mark) is not correct, based on the return value of method checkValidCharacter, the method will return - “Incorrect check mark!”.
checkPersonIDNumber	boolean	final String personID	**Private method.** The method will check the given string has **11** characters and that the “<u>century character</u>” in the string is a valid one, ‘**+**’, ‘**-**‘ or ‘**A’**.
checkLeapYear	boolean	int year	**Private method.** The method will check whether the given year is a leap year or not (true = leap year).
checkValidCharacter	boolean	final String personID	**Private method.** The method will receive a social security number as an input string. The method will check the last character of the person id is given correctly. Note – here, in this method, the value “**221199-123A**” can be set to be a valid test value (although not a valid

			value in real life). The formula for the calculation explained **here**. In your own IDE, test e.g., with your own social security number or some generated, valid social security number.
checkBirthdate	boolean	final String date	**Private method.** The method will receive the birthdate in the format of dd.mm.yyyy. The method will split the day, month and year from the given string and ensures the given year (positive value), the given month (1-12) and day (1-31) is within those valid ranges. In addition, if the previous conditions are met, the method will have to ensure the given days are valid for the given month (e.g., there are only 30 days in April). In the possible special case of February, the method will have to ensure the days are correct (28 or 29 depending on whether the year is a leap year or not). The method checkLeapYear is to be used here for this purpose

Course

Course – attributes

Attribute name	Type	Description / Notes
name	String	Name of the course.
courseCode	String	Code of the course, e.g., 811322A.
courseBase	Character	One of the values ‘A’, ‘P’ or ‘S’.
courseType	int	0 = Optional, 1 = Mandatory
period	int	1-5
credits	double	Valid values are MIN_CREDITS <= credits <= MAX_COURSE_CREDITS
numericGrade	boolean	Whether a course has numeric grading (true) or not (false).

Course - constructors

Constructor	Parameters	Description / Notes
Course	-	Parameter-less constructor (no functionality).
Course	String name, final int code, Character courseBase, final int type, final int period, final double credits, boolean numericGrade	The constructor must set the attribute values (and check for valid values for most of those – preferably by utilizing methods of the class (**setName**, **setCourseCode**, **setCourseType**, **setPeriod**, **setCredits**).
Course	Course course	The constructor must set the attribute values.

Course - methods

Method name	Return type	Parameters	Description / Notes
getName	String	-	Returns the course name.
setName	-	String name	The method will set the name of the course if the name is not empty or null.
getCourseTypeString	String	-	The method will return the string description of the course type (0 = “Optional” and 1 = “Mandatory”).
getCourseType	int	-	The method will return the type of the course (0 or 1).
setCourseType	-	final int type	The method will set the type if the value is either 0 or 1.
getCourseCode	String	-	The method will return the courseCode.
setCourseCode	-	final int courseCode, Character courseBase	If the courseCode is valid (0 < courseCode < 1000000) and courseBase is valid (‘A’, or ‘P’, or ‘S’), the method will set the courseCode as e.g., 811322A (concatenate the courseCode and courseBase) and set the courseBase.
getCourseBase	Character	-	The method will return the courseBase.
getPeriod	int	-	The method will return the period of the course.
setPeriod	-	final int period	The method will set the period if the given period is within the limits (MIN_PERIOD <= period <= MAX_PERIOD).
getCredits	double	-	The method will return the number of credits.
setCredits	-	final double credits	**Private method.** The method will set the credits if the given value is within the limits (MIN_CREDITS <= credits <= MAX_COURSE_CREDITS)

isNumericGrade	boolean	-	The method will return numericGrade.
setNumericGrade	-	boolean numericGrade	The method will set numericGrade.
toString	String	-	The method will output the expected output for an object of type Course. **See examples below ***)**

Example output *)**

```
[811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.]
[112233P (45.00 cr), "All kinds of basic studies". Mandatory, period: 2.]
[818181A (25.00 cr), "More master studies". Mandatory, period: 1.]
[666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.]
[919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.]
```

StudentCourse

StudentCourse – attributes

Attribute name	Type	Description / Notes
course	Course	A course a student has completed.
gradeNum	int	A grade of the student for the course. For a numerically graded course the range is 0-5, for a letter graded course the value can either be 'F' (failed) or 'A' (accepted).
yearCompleted	int	Year when the student has completed this course. The year must be within the range 2000 < yearCompleted <= currentYear.

StudentCourse - constructors

Constructor	Parameters	Description / Notes
StudentCourse		Parameter-less constructor (no functionality).
StudentCourse	Course course, final int gradeNum, final int yearCompleted	The constructor will set the given course, gradeNum and yearCompleted for the course. Preferably, for the gradeNum the method **setGrade** should be used.

StudentCourse - methods

Method name	Return type	Parameters	Description / Notes
getCourse	Course	-	Returns the course.
setCourse	-	Course course	The method will set the course.
getGradeNum	int	-	The method will return gradeNum.
setGrade	-	int gradeNum	**Protected method** (to be utilized from Main). The method will check whether the course is graded numerically or not. - If the course is graded numerically, the given value must be within the range (0 <= value <= 5, utilize method **checkGradeValidity**) - If the course has letter grading, the value must be either 'F' or 'A' (utilize method **checkGradeValidity**). - In addition, if yearCompleted has not been set, it will bet set to the current year.
checkGradeValidity	boolean	final int gradeNum	**Private method.** The method will check whether the given value is acceptable (0-5 or 'F' or 'A'). Only if the value is acceptable, the method will return true.
isPassed	boolean	-	The method will return false if the grade is 0 or 'F'.
getYear	int	-	The method will return yearCompleted.
setYear	-	final int year	The method will set the yearCompleted if the given value is within the range 2000 < year <= currentYear
toString	String	-	The method will output the expected output for an object of type StudentCourse. See examples below ***) **Note** – • if the grade is 0 (zero), the output for the grade will be "Not graded" (see below). Otherwise… • if the grade is numeric (the course has numeric grading), the numeric grade will be shown and

			• if the grade is a letter (‘A’ or ‘F’, i.e., the course is not numerically graded) the letter for the grade is shown, as in the **example below**.

*****)**

For a course which has **not** been graded (grade 0), the output would be…

```
[919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.] Year: 2021, Grade: "Not
graded".]
```

For a course which has numeric grading (1-5), the output would be…

```
[811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
```

For a course which has letter grading (‘F’ or ‘A’, in this case ‘A’)…

```
[666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.] Year: 2021, Grade: A.]
```

Degree

Degree – attributes

Attribute name	Type	Description / Notes
MAX_COURSES	static final int	50
count	int	Number of courses for this degree.
degreeTitle	String	By default, “No title”.
titleOfThesis	String	By default, “No title”.
myCourses	StudentCourse []	Array of StudentCourse objects. The array has default size MAX_COURSES.

Degree - methods

Method name	Return type	Parameters	Description / Notes
getCourses	StudentCourse []	-	Returns the courses of this degree.
addStudentCourses	-	StudentCourse [] courses	The method will add the courses to the degree, if the object is not null and courses can be added. The method should utilize the method **addStudentCourse** (see below).
addStudentCourse	boolean	StudentCourse course	The method will add the course to the degree, if the method is not null and the number of added courses is less than MAX_COURSES. The method will also udpate the value of count.
getDegreeTitle	String	-	The method returns the degreeTitle.
setDegreeTitle	-	String degreeTitle	The method will set the degreeTitle if the given string is not null.
getTitleOfThesis	String	-	The method will return the title of the thesis.
setTitleOfThesis	-	String titleOfThesis	The method will set the titleOfThesis is the given string is not null.
getCreditsByBase	double	Character base	The method will return the sum of the credits for all **completed** courses for a given **base** (‘P’, ‘A’ or ‘S’).
getCreditsByType	double	final int courseType	The method will return the sum of the credits for all **completed** courses for a given **type** (0 = optional , 1 = mandatory).
getCredits	double	-	The method will return the sum of all **completed** courses.
isCourseCompleted	boolean	StudentCourse c	**Private method.** The method will return the boolean value whether a course is completed or not. - If the course is not null and the course has <u>numeric</u> grading, the course is completed if the grade is **> 0**. - If the course is not null and the course has <u>letter</u> grading the course is completed if the grade is ‘**A**’. ⇨ Hint, utilize method **isPassed** (see **StudentCourse** class).

printCourses	-	-	The method will output all StudentCourses (which are not null).
toString	String	-	The method will output the expected output for an object of type Degree. **See examples below ***)**

*****)**

```
Degree [Title: "Bachelor of Science" (courses: 5)
        Thesis title: "Christmas - The most wonderful time of the year"
        1. [811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
        2. [112233P (45.00 cr), "All kinds of basic studies". Mandatory, period: 2.] Year: 2014, Grade: 1.]
        3. [223344A (50.50 cr), "More basic studies". Mandatory, period: 1.] Year: 2015, Grade: 1.]
        4. [556677A (50.00 cr), "Even more basic studies". Optional, period: 3.] Year: 2016, Grade: 4.]
        5. [123123A (50.50 cr), "Final basic studies". Mandatory, period: 4.] Year: 2017, Grade: 5.]]

Degree [Title: "Master of Science" (courses: 6)
        Thesis title: "Dreaming of a white Christmas"
        1. [616161A (25.00 cr), "Programming 2". Mandatory, period: 3.] Year: 2018, Grade: 1.]
        2. [717171P (45.00 cr), "All kinds of master studies". Optional, period: 2.] Year: 2019, Grade: 1.]
        3. [818181A (25.00 cr), "More master studies". Mandatory, period: 1.] Year: 2020, Grade: 2.]
        4. [919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.] Year: 2021, Grade: 3.]
        5. [666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.] Year: 2021, Grade: A.]
        6. [888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: F.]]

Degree [Title: "No title" (courses: 0)
        Thesis title: "No title"]
```

Student

Student – attributes

Attribute name	Type	Description / Notes
firstName	String	By default, “No name”.
lastName	String	By default, “No name”.
id	int	Generated randomly. Possible values 1-100.
startYear	int	Studies must have started after 2000 and the year cannot be in the future, i.e., 2000 < startYear <= current year. By default, current year.
graduationYear	int	To allow a student to have a graduation year set, the student must have **a)** all required credits completed (for both bachelor and masters studies & valid titles for both theses set, see below method descriptions) and **b)** the graduation year must be within the timeframe from the start of the studies to this date (cannot be before the start year or in the future).
degreeCount	int	Number of degrees (3)
degrees	Degree []	Array of the student’s degree studies. By default an array of three elements (0 = bachelor, 1 = master, 2 = reserved for doctoral studies).
birthDate	String	Date of birth in the format dd.mm.yyyy. By default, “Not available”.

Student - constructors

Constructor	Parameters	Description / Notes
Student	-	The id for the student will be set in the constructor. By default, the startYear is set to the currentYear. The array elements are created here.
Student	String lname, String fname	The id for the student will also be set in the constructor (utilize the above version of the constructor).

Student – methods (new methods highlighted below)

Method name	Return type	Parameters	Description / Notes
getFirstName	String	-	By default, returns "No name".
setFirstName	void	String firstName	Can be set only if the given name is not null.
getLastName	String	-	By default, returns "No name".
setLastName	void	String lastName	Can be set only if the given name is not null.
getId	int	-	The method will return the id of the student.
setId	void	final int id	Allowed values are 1-100, otherwise not set.
getstartYear	int	-	Returns the startYear (by default, current year, if not set).
setstartYear	-	final int startYear	The year must be after 2000 and cannot be in the future, i.e., 2000 < startYear <= current year.
getGraduationYear	int	-	
setGraduationYear *** Note the change of one of the return texts.	String	final int graduationYear	The graduation year can be set only if the student has **a)** completed all required credits (both bachelor & masters credits and titles set for the theses, utilize the method **canGraduate** here) and **b)** the given year is within the timeframe from the start of the studies to this date (neither before the start of the studies nor in the future). If… • a student cannot graduate (above case a), the method will return "**Check amount of required credits**". • the given year is invalid (above case b), the method will return "Check graduation year". • both cases are fine, the method will return "Ok".
setDegreeTitle	-	final int i, String dName	The method will set the title for the given degree (see class **Degree**) if the given index is valid (0 <= i < degreeCount) and the given title is not null.
addCourse	boolean	final int i, **StudentCourse** course	The method will add the course for the given degree (see class **Degree**) if the given index is valid (0 <= i < degreeCount) and the given course is not null.
addCourses	int	final int i, StudentCourse [] courses	The method will add the courses for the given degree (see class **Degree**) if the given index is valid (0 <= i < degreeCount) and the given array or single course is not null. The method will return the number of added courses.
printCourses	-	-	The method will print all the student's possible courses from all degrees (not null values). See classes **Degree** and **StudentCourse**.
printDegrees	-	-	The method will print all the student's degrees (not null values). See class **Degree**.
setTitleOfThesis	-	final int i, String title	The method will set the title for the given thesis (see class **Degree**) if the given index is valid (0 <= i < degreeCount) and the given title is not null.
getBirthDate	String	-	Returns the birthDate, by default "Not available".
setBirthDate	String	String personId	If the given person id is null or not valid, the method will return "No change".

			If the given person id is valid – the method will have to call setPersonID (and that method returns “Ok”), the attribute birthDate is set and the method returns the birthDate.
hasGraduated	boolean	-	Returns the information whether the student has graduated or not (based on the graduationYear).
canGraduate	boolean	-	**Private method.** The method will check the required studies for graduation, i.e., the student has enough of required credits (for both bachelor and masters) and the student has titles set for both theses (bachelor and masters). Use ConstantValues.BACHELOR_CREDITS and ConstantValues.MASTER_CREDITS.
getStudyYears	int	-	The method will check if the student has graduated or not and will return the number of years used for the studies.
getRandomId	int	-	**Private method.** The method will calculate a random integer for the range 1-100 (inclusive).
toString	String	-	The method will output the expected output for a student. **Note changes. ***)**

***)

```
Student id: 77
        First name: Donald, Last name: Duck
        Date of birth: "Not available"
        Status: The student has not graduated, yet
        Start year: 2001 (studies have lasted for 22 years)
        Total credits: 304.0
        Bachelor credits: 201.0
                Total bachelor credits completed (201.0/180.0)
                Title of BSc Thesis: "Bachelor thesis title"
        Master credits: 103.0
                Missing master's credits 17.0 (103.0/120.0)
                Title of MSc Thesis: "Masters thesis title"
```

```
Student id: 77
        First name: Donald, Last name: Duck
        Date of birth: 23.04.1998
        Status: The student has graduated in 2020
        Start year: 2001 (studies lasted for 19 years)
        Total credits: 324.0
        Bachelor credits: 201.0
                Total bachelor credits completed (201.0/180.0)
                Title of BSc Thesis: "Christmas - The most wonderful time of the year"
        Master credits: 123.0
                Total master's credits completed (123.0/120.0)
                Title of MSc Thesis: "Dreaming of a white Christmas""
```

Assessment and Grading

There will be deductions for…

- The code is expected to compile, otherwise the score will not be acceptable
- Each missing or incorrectly declared attribute (e.g., naming, visibility of type)
- Each missing required default value for an attribute
- Each missing or incorrectly declared constructor
- Each missing or incorrectly declared method
- Each method not handling parameters/attributes as required
- Other possible issues or bugs.

Expected output - tests

Note that id numbers are randomly generated.

Note that the first student, the second student etc., refer to the objects required for the program. You may name the variables in your test program as you like (e.g., student1, student2…).

Below you may find examples of expected output. Remember to test your program! The functionality of the methods must be consistent with the required descriptions.

Student – main program

1. Create a **student** using the constructor with no parameters
2. Create **11 Courses** with the following data (see the **Course** class and the constructor(s))…

   Course 1: name = "Programming 1", courseCode = 811104, courseBase = 'P', courseType = 1, period = 1, credits = 5.0, numericGrade = true
   Course 2: name = "All kinds of basic studies", courseCode = 112233, courseBase = 'P', courseType = 1, period = 2, credits = 45.0, numericGrade = true
   Course 3: name = "More basic studies", courseCode = 223344, courseBase = 'a', courseType = 1, period = 1, credits = 50.5, numericGrade = true
   Course 4: name = "Even more basic studies", courseCode = 556677, courseBase = 'a', courseType = 0, period = 3, credits = 50.0, numericGrade = true
   Course 5: name = "Final basic studies", courseCode = 123123, courseBase = 'A', courseType = 1, period = 4, credits = 50.5, numericGrade = true
   Course 6: name = "Programming 2", courseCode = 616161, courseBase = 'A', courseType = 1, period = 3, credits = 25.0, numericGrade = true
   Course 7: name = "All kinds of master studies", courseCode = 717171, courseBase = 'P', courseType = 0, period = 2, credits = 45.0, numericGrade = true
   Course 8: name = "More master studies", courseCode = 818181, courseBase = 'A', courseType = 1, period = 1, credits = 25.0, numericGrade = true
   Course 9: name = "Even more master studies ", courseCode = 919191, courseBase = 'S', courseType = 1, period = 3, credits = 20.0, numericGrade = true
   Course 10: name = "Extra master studies", courseCode = 666666, courseBase = 'S', courseType = 0, period = 5, credits = 8.0, numericGrade = false
   Course 11: name = "Final master studies", courseCode = 888888, courseBase = 'S', courseType = 1, period = 5, credits = 18.0, numericGrade = false

3. Create **StudentCourses** from the courses above with the following data (see the **StudentCourse** class and the constructor(s))…

   StudentCourse 1: course 1, 1, 2013
   StudentCourse 2: course 2, 1, 2014
   StudentCourse 3: course 3, 1, 2015
   StudentCourse 4: course 4, 4, 2016
   StudentCourse 5: course 5, 5, 2017
   StudentCourse 6: course 6, 1, 2018
   StudentCourse 7: course 7, 1, 2019
   StudentCourse 8: course 8, 2, 2020
   StudentCourse 9: course 9, 0, 2021
   StudentCourse 10: course 10, ‘A’, 2021
   StudentCourse 11: course 11, ‘f’, 2022

4. Create an **array** of **bachelor StudentCourses** from the first five StudentCourses
5. Create an **array** of **master StudentCourses** from the remaining six StudentCourses
6. Set the **degreeTitle** of the **bachelor** studies of the student to "**Bachelor of Science**"
7. Set the **degreeTitle** of the **master** studies of the student to "**Master of Science**"
8. Set the **title of the bachelor thesis** of the student to "**Bachelor thesis title**"
9. Set the **title of the master thesis** of the student to "**Masters thesis title**"
10. Add the bachelor courses (see step 4) to **bachelor studies** for the student
11. Add the master courses (see step 5) to **master studies** for the student
12. Set the start year of the student to **2001**
13. Set graduation year of the student to **2020**
14. Set the first name of the student to "**Donald**"
15. Set the last name of the student to "**Duck**"
16. Print the details of the student using **toString** method.
17. Set birthdate of the student to "**230498-045T**"
18. Set the title of the bachelor thesis of the student to "**Christmas - The most wonderful time of the year**"
19. Set the title of the master thesis of the student to "**Dreaming of a white Christmas**"
20. Print the degrees of the student (use method **printDegrees**)
21. Set the grade of the course **919191S** to **3** for the student
22. Print the details of the student using **toString** method.
23. Print the degrees of the student (use method **printDegrees**)
24. Print the courses of the student use method (**printCourses**)
25. Set the grade of the course **888888S** (the last StudentCourse course) of the student to '**X**'
26. Print the details of the StudentCourse **888888S**.
27. Set the grade of the course **888888S** (the last StudentCourse course) of the student to '**a**'
28. Print the details of the StudentCourse **888888S**.
29. Set the grade of the course **811104P** (the first StudentCourse course) of the student to **6**
30. Print the details of the StudentCourse **811104P**.
31. Set the grade of the course **811104P** (the first StudentCourse course) of the student to **5**
32. Print the details of the StudentCourse **811104P**.

Expected output

```
Student id: 87
        First name: Donald, Last name: Duck
        Date of birth: "Not available"
        Status: The student has not graduated, yet
        Start year: 2001 (studies have lasted for 22 years)
        Total credits: 304.0
        Bachelor credits: 201.0
                Total bachelor credits completed (201.0/180.0)
                Title of BSc Thesis: "Bachelor thesis title"
        Master credits: 103.0
                Missing master's credits 17.0 (103.0/120.0)
                Title of MSc Thesis: "Masters thesis title"

Degree [Title: "Bachelor of Science" (courses: 5)
        Thesis title: "Christmas - The most wonderful time of the year"
        1. [811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
        2. [112233P (45.00 cr), "All kinds of basic studies". Mandatory, period: 2.] Year: 2014, Grade: 1.]
        3. [223344A (50.50 cr), "More basic studies". Mandatory, period: 1.] Year: 2015, Grade: 1.]
        4. [556677A (50.00 cr), "Even more basic studies". Optional, period: 3.] Year: 2016, Grade: 4.]
        5. [123123A (50.50 cr), "Final basic studies". Mandatory, period: 4.] Year: 2017, Grade: 5.]]

Degree [Title: "Master of Science" (courses: 6)
        Thesis title: "Dreaming of a white Christmas"
        1. [616161A (25.00 cr), "Programming 2". Mandatory, period: 3.] Year: 2018, Grade: 1.]
        2. [717171P (45.00 cr), "All kinds of master studies". Optional, period: 2.] Year: 2019, Grade: 1.]
        3. [818181A (25.00 cr), "More master studies". Mandatory, period: 1.] Year: 2020, Grade: 2.]
        4. [919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.] Year: 2021, Grade: "Not
graded".]
        5. [666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.] Year: 2021, Grade: A.]
        6. [888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: F.]]

Degree [Title: "No title" (courses: 0)
        Thesis title: "No title"]


Student id: 87
        First name: Donald, Last name: Duck
        Date of birth: 23.04.1998
        Status: The student has not graduated, yet
        Start year: 2001 (studies have lasted for 22 years)
        Total credits: 324.0
        Bachelor credits: 201.0
                Total bachelor credits completed (201.0/180.0)
                Title of BSc Thesis: "Christmas - The most wonderful time of the year"
        Master credits: 123.0
                Total master's credits completed (123.0/120.0)
                Title of MSc Thesis: "Dreaming of a white Christmas"

Degree [Title: "Bachelor of Science" (courses: 5)
        Thesis title: "Christmas - The most wonderful time of the year"
        1. [811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
        2. [112233P (45.00 cr), "All kinds of basic studies". Mandatory, period: 2.] Year: 2014, Grade: 1.]
        3. [223344A (50.50 cr), "More basic studies". Mandatory, period: 1.] Year: 2015, Grade: 1.]
        4. [556677A (50.00 cr), "Even more basic studies". Optional, period: 3.] Year: 2016, Grade: 4.]
        5. [123123A (50.50 cr), "Final basic studies". Mandatory, period: 4.] Year: 2017, Grade: 5.]]

Degree [Title: "Master of Science" (courses: 6)
        Thesis title: "Dreaming of a white Christmas"
        1. [616161A (25.00 cr), "Programming 2". Mandatory, period: 3.] Year: 2018, Grade: 1.]
        2. [717171P (45.00 cr), "All kinds of master studies". Optional, period: 2.] Year: 2019, Grade: 1.]
        3. [818181A (25.00 cr), "More master studies". Mandatory, period: 1.] Year: 2020, Grade: 2.]
        4. [919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.] Year: 2021, Grade: 3.]
        5. [666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.] Year: 2021, Grade: A.]
        6. [888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: F.]]

Degree [Title: "No title" (courses: 0)
        Thesis title: "No title"]

[811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
[112233P (45.00 cr), "All kinds of basic studies". Mandatory, period: 2.] Year: 2014, Grade: 1.]
[223344A (50.50 cr), "More basic studies". Mandatory, period: 1.] Year: 2015, Grade: 1.]
[556677A (50.00 cr), "Even more basic studies". Optional, period: 3.] Year: 2016, Grade: 4.]
[123123A (50.50 cr), "Final basic studies". Mandatory, period: 4.] Year: 2017, Grade: 5.]
```

```
[616161A (25.00 cr), "Programming 2". Mandatory, period: 3.] Year: 2018, Grade: 1.]
[717171P (45.00 cr), "All kinds of master studies". Optional, period: 2.] Year: 2019, Grade: 1.]
[818181A (25.00 cr), "More master studies". Mandatory, period: 1.] Year: 2020, Grade: 2.]
[919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.] Year: 2021, Grade: 3.]
[666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.] Year: 2021, Grade: A.]
[888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: F.]

[888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: F.]
[888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: A.]
[811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
[811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 5.]
```

Student – main program – missing credits

1. Create a **student** using the constructor with no parameters
2. Create **11 Courses** with the following data (see the **Course** class and the constructor(s))…

   Course 1: name = "Programming 1", courseCode = 811104, courseBase = 'P', courseType = 1, period = 1, credits = 5.0, numericGrade = true
   Course 2: name = "All kinds of basic studies", courseCode = 112233, courseBase = 'P', courseType = 1, period = 2, credits = 45.0, numericGrade = true
   Course 3: name = "More basic studies", courseCode = 223344, courseBase = 'a', courseType = 1, period = 1, credits = 50.5, numericGrade = true
   Course 4: name = "Even more basic studies", courseCode = 556677, courseBase = 'a', courseType = 0, period = 3, credits = 50.0, numericGrade = true
   Course 5: name = "Final basic studies", courseCode = 123123, courseBase = 'A', courseType = 1, period = 4, credits = 50.5, numericGrade = true
   Course 6: name = "Programming 2", courseCode = 616161, courseBase = 'A', courseType = 1, period = 3, credits = 25.0, numericGrade = true
   Course 7: name = "All kinds of master studies", courseCode = 717171, courseBase = 'P', courseType = 0, period = 2, credits = 45.0, numericGrade = true
   Course 8: name = "More master studies", courseCode = 818181, courseBase = 'A', courseType = 1, period = 1, credits = 25.0, numericGrade = true
   Course 9: name = "Even more master studies ", courseCode = 919191, courseBase = 'S', courseType = 1, period = 3, credits = 20.0, numericGrade = true
   Course 10: name = "Extra master studies", courseCode = 666666, courseBase = 'S', courseType = 0, period = 5, credits = 8.0, numericGrade = false
   Course 11: name = "Final master studies", courseCode = 888888, courseBase = 'S', courseType = 1, period = 5, credits = 18.0, numericGrade = false

3. Create **StudentCourses** from the courses above with the following data (see the **StudentCourse** class and the constructor(s))…

   StudentCourse 1: course 1, 1, 2013
   StudentCourse 2: course 2, 1, 2014
   StudentCourse 3: course 3, 1, 2015
   StudentCourse 4: course 4, 4, 2016
   StudentCourse 5: course 5, 5, 2017
   StudentCourse 6: course 6, 1, 2018
   StudentCourse 7: course 7, 1, 2019
   StudentCourse 8: course 8, 2, 2020
   StudentCourse 9: course 9, 0, 2021
   StudentCourse 10: course 10, ‘A’, 2021
   StudentCourse 11: course 11, ‘f’, 2022

4. Create an **array** of **bachelor StudentCourses** from the **first three** StudentCourses
5. Create an **array** of **master StudentCourses** from the **last three** StudentCourses
6. Set the **degreeTitle** of the **bachelor** studies of the student to "**Bachelor of Science**"
7. Set the **degreeTitle** of the **master** studies of the student to "**Master of Science**"
8. Set the **title of the bachelor thesis** of the student to "**Bachelor thesis title**"
9. Set the **title of the master thesis** of the student to "**Masters thesis title**"
10. Add the bachelor courses (see step 4) to **bachelor studies** for the student
11. Add the master courses (see step 5) to **master studies** for the student
12. Set the start year of the student to **2001**
13. Set graduation year of the student to **2020**
14. Set the first name of the student to "**Donald**"
15. Set the last name of the student to "**Duck**"
16. Print the details of the student using **toString** method.
17. Set birthdate of the student to "**230498-045T**"
18. Set the title of the bachelor thesis of the student to "**Christmas - The most wonderful time of the year**"
19. Set the title of the master thesis of the student to "**Dreaming of a white Christmas**"
20. Print the details of the student using **toString** method.
21. Print the degrees of the student (use method **printDegrees**)

Expected output – missing credits

```
Student id: 86
        First name: Donald, Last name: Duck
        Date of birth: "Not available"
        Status: The student has not graduated, yet
        Start year: 2001 (studies have lasted for 22 years)
        Total credits: 108.5
        Bachelor credits: 100.5
                Missing bachelor credits 79.5 (100.5/180.0)
                Title of BSc Thesis: "Bachelor thesis title"
        Master credits: 8.0
                Missing master's credits 112.0 (8.0/120.0)
                Title of MSc Thesis: "Masters thesis title"


Student id: 86
        First name: Donald, Last name: Duck
        Date of birth: 25.04.1904
        Status: The student has not graduated, yet
        Start year: 2001 (studies have lasted for 22 years)
        Total credits: 108.5
        Bachelor credits: 100.5
                Missing bachelor credits 79.5 (100.5/180.0)
                Title of BSc Thesis: "Christmas - The most wonderful time of the year"
        Master credits: 8.0
                Missing master's credits 112.0 (8.0/120.0)
                Title of MSc Thesis: "Dreaming of a white Christmas"


Degree [Title: "Bachelor of Science" (courses: 3)
        Thesis title: "Christmas - The most wonderful time of the year"
        1. [811104P ( 5.00 cr), "Programming 1". Mandatory, period: 1.] Year: 2013, Grade: 1.]
        2. [112233P (45.00 cr), "All kinds of basic studies". Mandatory, period: 2.] Year: 2014, Grade: 1.]
        3. [223344A (50.50 cr), "More basic studies". Mandatory, period: 1.] Year: 2015, Grade: 1.]]

Degree [Title: "Master of Science" (courses: 3)
        Thesis title: "Dreaming of a white Christmas"
        1. [919191S (20.00 cr), "Even more master studies". Mandatory, period: 3.] Year: 2021, Grade: "Not
graded".]
        2. [666666S ( 8.00 cr), "Extra master studies". Optional, period: 5.] Year: 2021, Grade: A.]
        3. [888888S (18.00 cr), "Final master studies". Mandatory, period: 5.] Year: 2022, Grade: F.]]

Degree [Title: "No title" (courses: 0)
        Thesis title: "No title"]
```